A Brief Introduction to Entity Framework Core

Back to: ASP.NET Core Web API Tutorials

Introduction to Entity Framework Core

Entity Framework Core (EF Core) is a modern, lightweight, and cross-platform Object-Relational Mapper (ORM) developed by Microsoft for the .NET ecosystem. It enables developers to interact with databases using C# objects and LINQ queries instead of writing raw SQL commands. EF Core simplifies data access, improves productivity, and provides a clean abstraction between the application and the database, making it ideal for building modern, scalable, and cloud-ready applications.

What is Entity Framework Core?

Entity Framework Core (EF Core) is Microsoft's **Object-Relational Mapper (ORM)** for .NET applications. It acts as a **bridge between our C# code and the underlying database**, allowing us to perform all database operations (such as creating tables, inserting, updating, deleting, and retrieving data) using **C# objects and LINQ queries**, without writing SQL manually.

EF Core is:

- **Open-Source**: Free and community-driven, maintained by Microsoft and contributors on GitHub.
- **Lightweight**: Designed to be modular and optimized for performance.
- **Cross-Platform**: Works seamlessly on **Windows, Linux, and macOS** with **.NET Core** (for cross-platform apps) and **.NET Framework 4.6+**.
- **Extensible**: Developers can add their own conventions, behaviours, or plugins as needed.

So, instead of manually managing Connections, Commands, and SQL Scripts, developers focus on writing business logic while EF Core **handles all low-level database communication** in the background.

What is ORM?

ORM (Object-Relational Mapping) is a programming technique that **maps database (such as SQL Server, MySQL, Oracle, etc.) tables to programming-language (such as C#, Java, etc.) classes** and **rows to class objects**. In simpler terms:

- Each **Table** in the database corresponds to a **Class** in our code.
- Each **Column** becomes a **Property** in that class.
- Each **Record (Row)** becomes an **Object (Instance)** of that class.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The ORM framework:

- Automatically generates **SQL commands** to perform **CRUD operations** (Create, Read, Update, Delete).
- Handles **Opening and Closing Connections**, **Transactions**, and **Error Management**.
- Converts **Relational Data** (from SQL) into **Object-Oriented Structures** (C# classes) and vice versa.

Example: Without ORM, we would manually write INSERT INTO, SELECT, UPDATE, and DELETE SQL statements. With ORM like EF Core, it will automatically generate and execute the appropriate SQL under the hood.

Why Do We Need to Use an ORM Framework?

Without an ORM, developers must:

1. Write SQL for each CRUD operation.
2. Map query results manually to objects.
3. Handle connections and transactions.
4. Maintain consistency between the database schema and the application code.

This process is repetitive, time-consuming, and error-prone. An ORM like **Entity Framework Core**:

- Automates these tasks.
- Reduces repetitive code.
- Improves maintainability.
- Allows developers to focus on **business logic** rather than **data access logic**.

In short, ORM frameworks act as a **Translator** and **Manager** between your application and the database, freeing you from writing low-level data access logic repeatedly.

Why Should We Use ORM Like Entity Framework Core?

The following are some reasons to use an ORM like Entity Framework Core in your applications.

- **Simplifies Data Access:** You interact with your data through C# classes and LINQ queries instead of raw SQL. EF Core automatically translates these queries into efficient SQL commands.
- **Reduces Repetitive Code:** No need to write connection code, parameterized SQL, or command objects. EF Core handles these tasks automatically.
- **Increases Productivity:** Developers can focus on logic and features rather than repetitive SQL code, significantly speeding up development.
- **Cross-Platform Compatibility:** EF Core works on all major operating systems and database providers, making your app portable and modern.
- **Improves Performance:** EF Core supports features like:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

    - **Lazy Loading** – Loads data only when needed.
    - **Eager Loading** – Loads related data upfront to reduce queries.
    - **Caching** – Reuses tracked data efficiently within the context.
- **Ensures Data Integrity:** It provides built-in **transaction management**, **concurrency control**, and **validation mechanisms**, ensuring reliable and consistent data handling.

Why We Need to Use Entity Framework Core over EF 6.x?

Entity Framework 6.x (often called EF 6) was the classic version of Microsoft's Object-Relational Mapping (ORM) framework, designed to work with the .NET Framework on Windows. Entity Framework Core (EF Core) is the next-generation, lightweight, and cross-platform version of Entity Framework, built from the ground up to support modern .NET applications running on .NET.

Although EF 6.x is still functional, it has reached maturity and is in maintenance mode, meaning no major new features are being added. In contrast, EF Core continues to evolve with each .NET release, providing better performance, flexibility, and modern capabilities. Let's understand the major reasons why EF Core is preferred over EF 6.x.

Cross-Platform and Cloud-Ready

Older frameworks like Entity Framework 6 worked only on Windows because they relied on the traditional .NET Framework. EF Core, however, is built on **.NET Core**, which is **Cross-Platform**. This means you can now:

- Build and run your applications on **Windows, macOS, or Linux**.
- Host your web APIs on **Docker Containers or Cloud Environments** (e.g., Azure, AWS, GCP).
- Use the same EF Core codebase across multiple environments, from local development to cloud deployment.

Example: You can develop an ASP.NET Core Web API on Windows and deploy it on Linux-based servers without changing any data access code. This flexibility was not possible with older EF versions. In short, EF Core enables truly portable applications that can be deployed anywhere, on-premises or in the cloud.

Modular and Lightweight Architecture

EF 6.x is designed based on Monolithic Architecture, i.e., all features came bundled together, even if you didn't need them. On the other hand, EF Core is based on Modular Architecture, i.e., you only need to add the features or database providers you actually use, through NuGet packages. This modular structure makes EF Core:

- Easier to maintain.
- Faster to start up.
- More suitable for microservice and modular architectures.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Example: With EF Core, you can install just Microsoft.EntityFrameworkCore.SqlServer if you use SQL Server, without pulling unnecessary dependencies.

Extensible and Customizable Framework

EF Core is designed around a **Lightweight and Modular Architecture**. You can easily extend its behaviour by adding:

- **Custom Conventions** (e.g., Naming Rules for Tables or Columns)
- **Third-party Database Providers** (e.g., PostgreSQL, MySQL, MongoDB)
- Use **Interceptors** to track queries or log SQL commands before execution.
- **Third-Party Plugins (Z.EntityFramework.Extensions.EFCore, EFCore.BulkExtensions, etc.)** for batch operations to get the best performance**.**

EF 6.x had limited extensibility and customization options compared to EF Core's flexible, modern design. In short, EF Core is designed to be developer-friendly and easily extensible for real-world business applications.

High Performance and Efficiency

Performance was a major limitation in older EF versions, but EF Core has been **rewritten from Scratch** with performance in mind, not just an upgrade of EF 6. This redesign introduced significant performance and efficiency gains, such as:

- **Improved Query Translation**: LINQ queries are converted to optimized SQL.
- **Batch Command Execution**: Combines multiple **INSERT/UPDATE/DELETE** statements into one round trip to the database.
- **Change Tracking Optimization**: Detects modified entities efficiently to update only what's necessary.
- **Compiled Queries**: Caches the SQL generated from frequently executed queries for faster subsequent execution.

All these improvements make EF Core significantly **Faster** than Entity Framework 6.x in most real-world scenarios. Real-world tests show EF Core often performs **2x–5x Faster** than Entity Framework 6.x. In Short, EF Core applications generally perform better, especially when dealing with large datasets or high-frequency transactions.

Example: When inserting 1,000 records, EF 6.x would send 1,000 INSERT statements individually. EF Core batches them together and sends a single combined batch, reducing round-trips and improving performance.

Continuous Development and Active Community Support

EF 6.x is in maintenance mode, receives only security and compatibility updates. On the other hand, EF Core is actively developed with new features, bug fixes, and performance enhancements in every .NET release. Microsoft

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

continues to invest heavily in EF Core, introducing features like:

- Temporal Tables.
- Bulk Operations.
- Improved Raw SQL Mapping.
- Better Cosmos DB and NoSQL support.

Conclusion: EF Core is the future of Entity Framework, continuously evolving to meet modern application needs.

Flexible and Database-Agnostic

EF Core is not tied to any single database engine. It supports multiple **Database Providers**, allowing you to switch from one database to another (e.g., SQL Server to PostgreSQL) with minimal changes. Supported databases include:

- Microsoft SQL Server
- PostgreSQL
- MySQL
- SQLite
- Oracle
- Cosmos DB (NoSQL)

This flexibility makes EF Core ideal for modern **Multi-Database Applications** or **Microservice Architectures**, where different services might use different databases but share the same data-access framework.

- **Scenario-1:** Your ProductService might use PostgreSQL for scalability, while your PaymentService uses SQL Server; both can use EF Core with just different providers.
- **Scenario-2:** You can start a project using SQLite for development (lightweight and file-based) and easily migrate to SQL Server or PostgreSQL in production without rewriting your data access layer.

Modern and Developer-Friendly Features

EF Core includes the modern .NET ecosystem with rich features that make development smoother and safer:

- **Asynchronous Programming (async/await):** Prevents blocking I/O during long database operations, improving app responsiveness and scalability.
- **LINQ Support:** Allows writing strongly-typed, compile-time-safe queries using C# syntax instead of SQL strings.
- **Shadow Properties:** Enables tracking fields not defined in your entity class (e.g., timestamps).
- **Global Query Filters:** Automatically apply conditions (e.g., soft delete or tenant isolation) across all queries.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- **Value Converters:** Transform data between .NET types and database types (e.g., enum ↔ string, bool ↔ int).

Suitable for Modern Architectures

Today's software architectures, such as microservices, cloud-native applications, and cross-platform APIs, require lightweight, fast, and flexible frameworks. EF Core fits this modern development ecosystem perfectly:

- Works with ASP.NET Core Web API and Minimal APIs.
- Integrates seamlessly with Azure, Docker, and Kubernetes.
- Allows per-service data models in a microservices architecture.

EF 6.x, by contrast, was designed for older, monolithic .NET Framework applications. In short, EF Core is built for modern, distributed, and scalable systems.

Entity Framework Core Development Approaches:

Entity Framework Core (EF Core) provides two primary development approaches to define and manage the relationship between an application's data model and its underlying database:

1. **Code-First Approach**
2. **Database-First Approach**

Both approaches achieve the same ultimate goal, synchronizing your application's data model with a database, but they differ in **where the design starts** (in code or in the database) and **how the database and model stay in sync**. Let's understand each approach in detail.

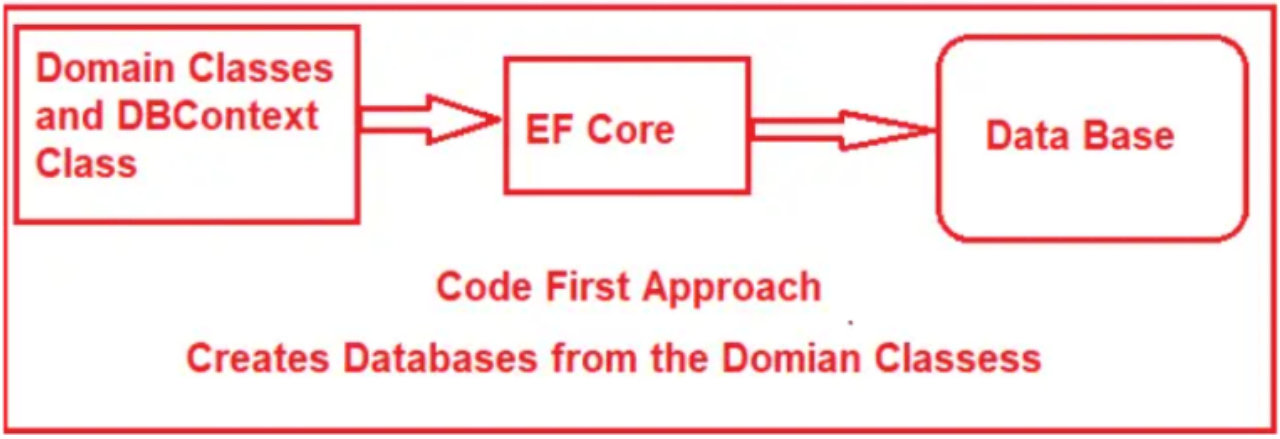
Entity Framework Core Code First Approach:

The **Code-First Approach** starts from the **application code**. Here, developers design and define the data model, i.e., all entity classes such as Student, Department, Address, etc., within the .NET application. These classes represent the structure of the data the application will manage.

Once the domain model is ready, EF Core uses this model to **automatically generate the database schema** (tables, relationships, keys, and constraints). This process is known as **database migration**.

In short, you design the classes → EF Core interprets them → EF Core builds the database schema accordingly. For a better understanding, please have a look at the following diagram.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



When to Use Code-First

The Code-First Approach is ideal when:

- The **Application is New**, and no existing database is available.
- The development team wants **Full Control** over the database schema design.
- The database is meant to evolve along with the application model (through code changes and migrations).
- You prefer to maintain the **Database Schema Inside Source Control** (since model changes are tracked in code).

Advantages

- Gives developers **Complete Control** over model design.
- Encourages **Domain-Driven Design** principles (focus on business objects rather than database tables).
- Simplifies schema evolution through **Migrations**.
- Keeps the database model in sync with the application logic automatically.

Limitations

- Not ideal for projects where the database already exists or is maintained by another team.
- Requires careful migration management to prevent accidental schema changes in production.
- Changes to class models require migrations to update the database schema.

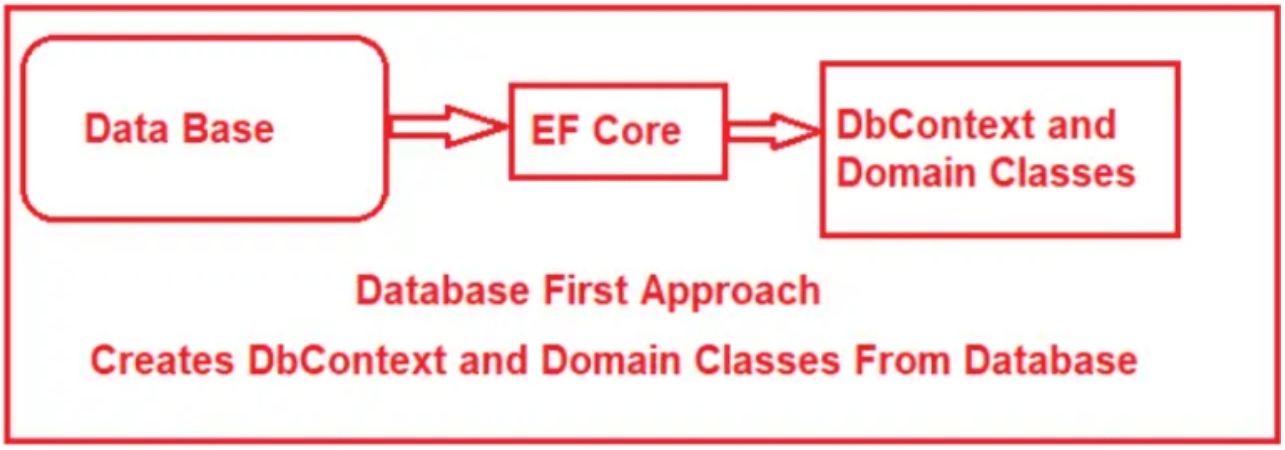
Entity Framework Core Database First Approach:

The **Database-First Approach** is the reverse of the Code-First Approach. In this approach, the starting point is an **Existing Database**, one that might already have tables, relationships, stored procedures, and constraints in place.

Entity Framework Core reads the existing database schema and automatically **Generates Corresponding Domain Classes** (entities) and a **DbContext** class based on that schema. This process is often referred to as **Reverse Engineering**.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In short, we already have the database → EF Core reads it → EF Core generates matching C# entity classes and DbContext. For a better understanding, please have a look at the following diagram.



When to Use Database-First

The Database-First Approach is ideal when:

- You are working with an **Existing or Legacy Database**.
- A Separate Team manages the database structure, and you cannot modify it directly.
- Your application must integrate with **an established enterprise system** or **third-party database**.
- You want to **quickly generate data models** from an existing schema to start building the application.

Advantages

- Saves time when a database already exists.
- Ensures **100% Schema Consistency** between the application model and the database.
- Suitable for **Integration Projects** and **Enterprise Systems** with predefined database standards.
- Eliminates the need to manually design models.

Limitations

- Developers have **Less Control** over the database schema.
- Any database change requires **Manual Regeneration** of classes.
- Custom configurations (such as naming conventions or shadow properties) might be overwritten when the model is regenerated.
- The database-first approach can become cumbersome in **agile or rapidly evolving projects**, where schema changes are frequent.

Choose the Correct Approach:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Both approaches serve the same purpose, bridging your application and the database, but they differ in **direction and ownership**:

- **Code-First**: Starts from **code**, generates the **database**, and evolves through migrations.
  Ideal when **you design both the model and the database**. Choose **Code-First** when you're *building a new system from scratch*.
- **Database-First**: Starts from an **existing database**, generates **code**, and syncs with it.
  Ideal when **the database already exists** or is maintained by another team. Choose **Database-First** when you're *integrating with an existing system*.

In simple terms:

- **Code-First** = "Let my code define the database."
- **Database-First** = "Let my database define the code."

Entity Framework Core Database Providers:

Entity Framework Core (EF Core) is designed to be **Database-Agnostic**, meaning it doesn't directly know how to communicate with a specific database engine (like SQL Server or MySQL). Instead, EF Core uses a **Database Provider**, a specialized software component that acts as a **Bridge** between EF Core and a particular database system.

In simpler terms:

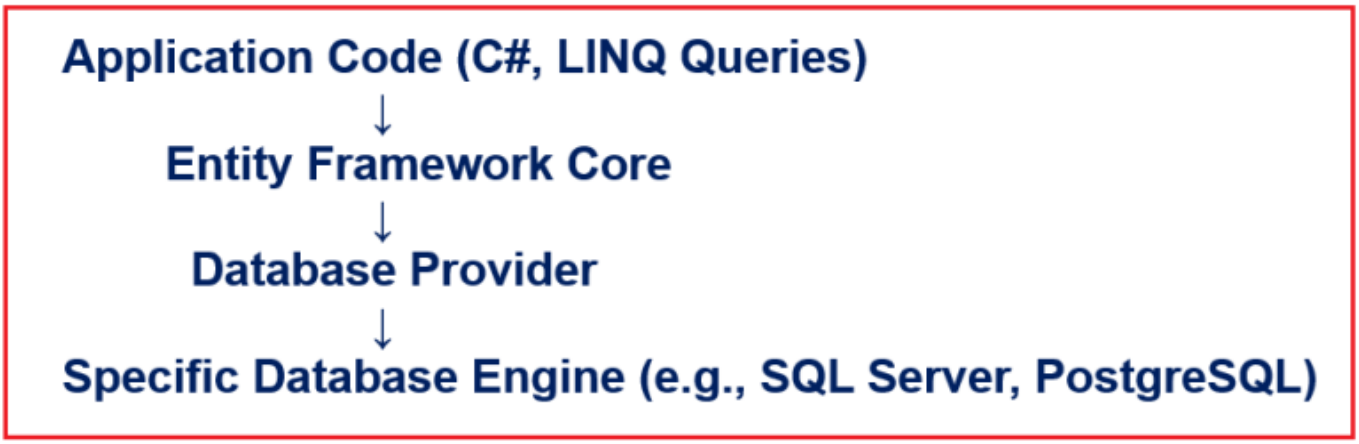
- The Database Provider translates the **Standard EF Core Commands** (such as queries, updates, and migrations) into **Database-Specific Commands** (SQL syntax or operations) that the underlying database understands.

So, EF Core remains **Generic and Consistent** across all databases, while the provider handles **differences in SQL commands, data types, and features**. So, the **provider acts as a bridge** between the EF Core framework and the database engine.

How the Database Provider Fits into EF Core Architecture

EF Core sits between your application and the database. For a better understanding, please have a look at the following image.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Here's the logical flow:

- The **Application Code** defines entities, relationships, and LINQ queries.
- **EF Core** processes these queries and passes them to the **Database Provider**.
- The **Database Provider** translates EF Core's query structure into database-specific SQL statements.
- Finally, the **Database Engine** executes those SQL commands and returns the results.

This design ensures **Separation of Concerns**. EF Core handles ORM functionality, while each provider focuses on the specifics of its database engine.

Why EF Core Uses Providers

Each database management system (DBMS) has its **own SQL syntax**, **data types**, and **capabilities**. For example:

- SQL Syntax (e.g., SQL Server uses GETDATE() for current date/time, while MySQL uses NOW())
- Data types (e.g., nvarchar vs text)
- Indexing mechanisms
- Transaction handling and concurrency models

A **single EF Core library cannot support all these variations natively**. Therefore, Microsoft and third-party developers create **Database Providers** that:

- Understand the syntax and data types of their database.
- Adapt EF Core's generic queries into optimized SQL for that database.
- Implement features such as migrations, connection management, and transactions in a way that fits the particular DBMS.

This allows using the same EF Core codebase across multiple databases by switching the provider package and connection string.

Commonly Used EF Core Database Providers

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The following are the most widely used EF Core providers:

Microsoft SQL Server

- **Description:** Default provider developed by Microsoft; ideal for enterprise-grade applications.
- **NuGet Package:** Microsoft.EntityFrameworkCore.SqlServer
- **Highlights:**
  - Excellent integration with .NET and Azure services.
  - Supports transactions, migrations, and complex queries.
  - Commonly used in enterprise and API-based applications.

SQLite

- **Description:** Lightweight, file-based database. No server installation required.
- **NuGet Package:** Microsoft.EntityFrameworkCore.Sqlite
- **Highlights:**
  - Perfect for small apps, testing, or mobile applications.
  - Data is stored in a single .db file.
  - Limited concurrency and scalability compared to server-based systems.

MySQL

- **Description:** Popular open-source relational database widely used for web applications.
- **NuGet Package:** Pomelo.EntityFrameworkCore.MySql (community-supported provider).
- **Highlights:**
  - Great for open-source and LAMP stack environments.
  - High performance for read-heavy workloads.
  - Often used with PHP and .NET Core web applications.

PostgreSQL

- **Description:** A robust, enterprise-class open-source RDBMS known for standards compliance and advanced features.
- **NuGet Package:** Npgsql.EntityFrameworkCore.PostgreSQL
- **Highlights:**
  - Excellent for complex queries, analytics, and JSON storage.
  - Supports advanced features like full-text search and custom data types.
  - Widely used in microservice and cloud-native architectures.

Oracle Database

- **Description:** High-performance, enterprise-level relational database.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- **NuGet Package:** Oracle.EntityFrameworkCore
- **Highlights:**
  - Ideal for large, mission-critical applications.
  - Offers strong security, high availability, and transaction consistency.
  - Commonly used in financial, healthcare, and government sectors.

MongoDB

- **Description:** A NoSQL, document-oriented database that stores data in flexible JSON-like documents.
- **NuGet Package:** MongoDB.EntityFrameworkCore
- **Highlights:**
  - Schema-less, great for rapidly evolving applications.
  - Scales horizontally and handles large volumes of unstructured data.
  - Often used for real-time analytics, IoT, and big data systems.

EF Core primarily targets **relational databases**, but extensions and third-party packages have made it compatible with **non-relational (NoSQL)** systems as well. For the complete list of EF Core Database Providers, please visit the following URL: **https://learn.microsoft.com/en-us/ef/core/providers/**

Entity Framework Core offers a powerful and flexible approach to data management in .NET applications. Its cross-platform support, high performance, and modular design make it the preferred choice for developers building modern enterprise and cloud-based solutions. By automating most database operations and supporting multiple databases through providers, EF Core streamlines development and helps maintain clean, maintainable, and efficient code.

Registration Open – Mastering Design Patterns, Principles, and Architectures using .NET

New Batch Starts: 11th March, 2026

Session Time: 6:30 AM – 08:00 AM IST

Advance your career with our expert-led, hands-on live training program. Get complete course details, the syllabus, and Zoom credentials for demo sessions via the links below.

View Course Details & Get Demo Credentials

Registration Form

Join Telegram Group

Join WhatsApp Group

Contact: +91 70218 01173 (Call / WhatsApp)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent | Do Not Sell My Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF